

CMPT 211: Introduction to Software Development

Comprehensive Midterm Study Guide

Concordia University of Edmonton

Winter 2026

Core Topics to Review

This exam focuses on the foundational concepts of software development in Python covered in the first half of the course. You are expected to not only know the definitions but also understand the "why" and "how" behind these architectures and engineering principles.

1. Language Architecture: How Python Runs

- **Compiled Languages (e.g., C, C++):** These languages require a compiler to translate the entire human-readable source code into machine code (binary) specifically tailored for the target CPU architecture *before* the program can run. They are fast but platform-dependent.
- **Interpreted Languages & The PVM (Python Virtual Machine):** Python is an interpreted language. Instead of compiling down to machine code directly, Python source code is first translated into an intermediate language called "bytecode." The PVM then acts as a software-based CPU, reading and executing this bytecode line-by-line. This is what makes Python platform-independent ("write once, run anywhere"), though it trades off some execution speed compared to compiled languages.

[Image of Python Virtual Machine architecture diagram]

2. Primitive Types vs. Elementary Data Structures

- **Primitive Types:** The absolute foundation of data.
 - **int:** Whole integers (e.g., 42, -5).
 - **float:** Floating-point decimal numbers (e.g., 3.14, 0.001).
 - **string:** Text characters wrapped in quotes (e.g., "Hello").
- **Lists (The "Backpack"):** A mutable (changeable), ordered sequence of elements. Lists are dynamic; Python allocates extra memory in the background so you can append, remove, or alter items after the list is created. Accessed via an integer index starting at 0.
- **Tuples (The "Sealed Box"):** An immutable (unchangeable), ordered sequence. Once created in memory, a tuple is locked. Attempting to reassign a value (e.g., `my_tuple[0] = 5`) will cause a crash. They are mathematically safer and computationally faster than lists, ideal for fixed coordinates or configurations.

- **Dictionaries / Maps (The "Tree Structure"):** A mutable data structure that maps a specific "Key" to a "Value". Unlike lists that rely on ordered numerical indexes, dictionaries allow you to access data via logical names (e.g., `student["grades"]`). Because values can be other dictionaries, they form nested tree-like structures.

3. Functions and Execution Flow

- **Definition of a Function:** A reusable, self-contained block of code designed to perform a single, specific task. Functions prevent code duplication and allow inputs to be transformed into outputs predictably.
- **Function Signature:** The blueprint of the function. It consists of the function's unique name and the specific parameters (inputs) it requires to operate. For example, in `def calculate_area(width, height):`, the signature dictates that the system must provide two arguments to invoke the logic.
- **The Call Stack:** A fundamental data structure (operating on a Last-In, First-Out or LIFO principle) that the Python interpreter uses to keep track of execution. When Function A calls Function B, execution of A pauses. Function A is placed on the "stack" along with its current memory state. Once Function B finishes and hits a `return` statement, it is popped off the stack, and Function A resumes exactly where it left off.

[Image of Call Stack LIFO memory diagram]

4. Core Software Engineering Principles

- **Modularity:** The practice of subdividing a computer program into separate, independent, and interchangeable modules (like Lego bricks).
- **Shallow, Modular Functions:** A best practice where complex logic is broken down into a "stack" of small functions. Each function should ideally do only one thing. This makes the code easier to test, debug, and read.
- **Robust vs. Fragile Engineering:**
 - *Fragile* code assumes perfect conditions. If a file is missing or a user types a letter instead of a number, the entire program crashes uncontrollably.
 - *Robust* code anticipates failure. Using "Inner Shields" like `try/except` blocks allows the program to catch errors (Exceptions), handle them gracefully (e.g., skipping a corrupt line in a CSV), and continue running.
- **Interpretability:** Code is read by humans much more often than it is written. Interpretability is the measure of how easily another developer can understand the intent, logic flow, and data transformations within your script without needing excessive external documentation.
- **OOP vs. Procedural Programming:** Procedural programming processes data in a top-down, step-by-step manner where data (variables) and behaviors (functions) are entirely separate. Object-Oriented Programming (OOP) encapsulates both the data (attributes) and the behaviors that manipulate that data (methods) into a single entity known as an "Object."